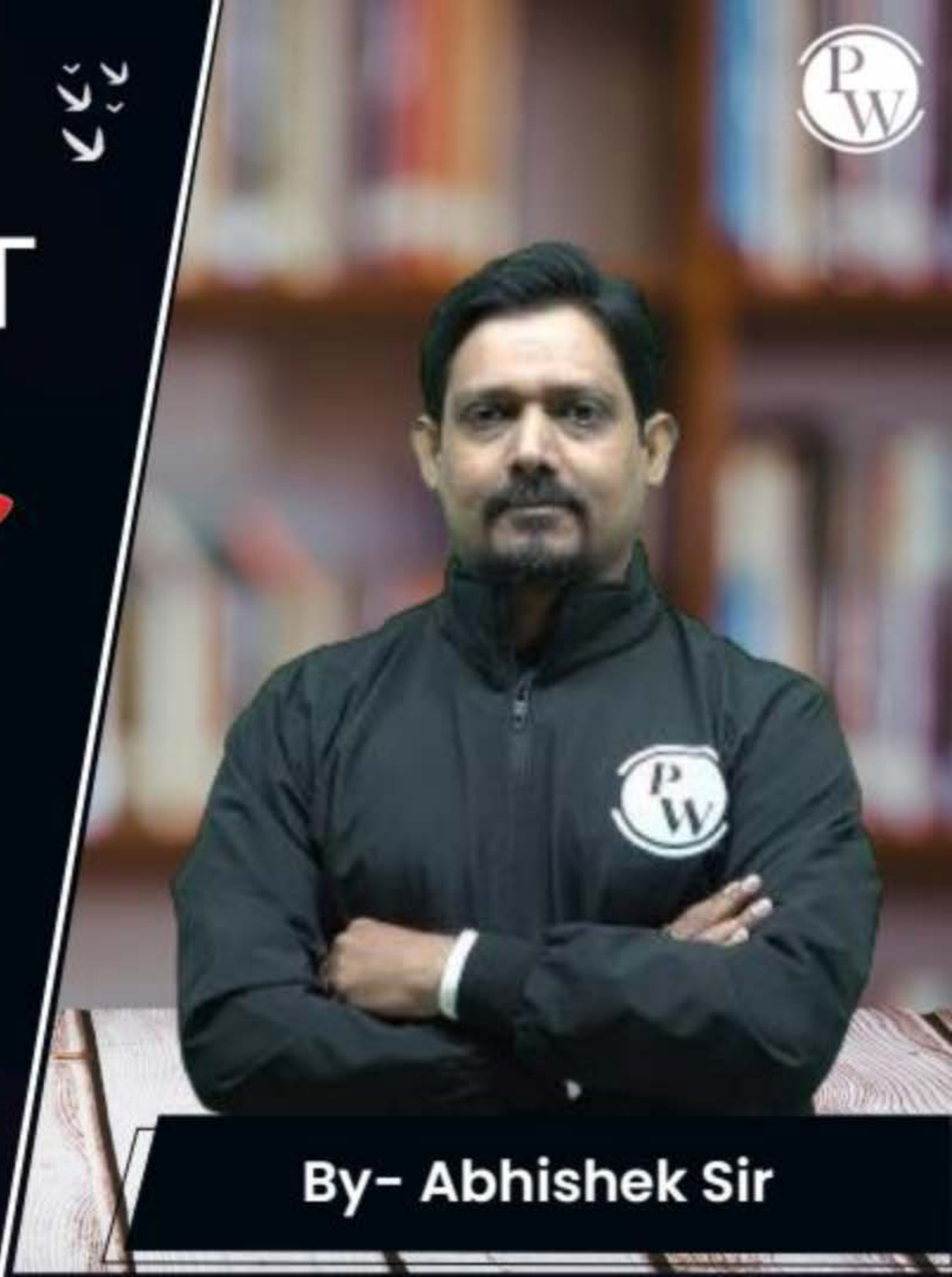
# Computer Science & IT



## C Programming

## Functions & storage classes

Lecture No. 04



By- Abhishek Sir

# Recap of Previous Lecture

30% Aph + EM + DM



- Topic: Activation tree
- Topic: Storage class
- Topic
- Topic
- Topic

Slide

3 Question 4-5 mark

# Topics to be Covered

3 Q : Data structure - 4-5 mark



- Topic — Storage class
- Topic — Recursion
- Topic
- Topic
- Topic

10% weightage

PDS

Slide



# GATE 2017

#Q The value of j at the end of the execution of the following C program

```
int incr (int i)
{
    static int count = 0;
    count = count + i;
    return (count);
}
main () {
    int i,j;
    for (i = 0; i <=4; i++)
      j = incr(i);
}
```

is

(a) 10

(b) 4

(c) 6

(d) 7

count

| 0 | 1 | 3 | 6 | 10 |
|---|---|---|---|---|

main ← first fun to call.

i = 0 j = incr(0) = 0

i = 1 j = incr(1) = 1

i = 2 j = incr(2) = 3

i = 3 j = incr(3) = 6

i = 4 j = incr(4) = 10

Slide

static default storage for global variable

Local can also be static and initialized only once

Automatically initialized to '0'

```c
#include <stdio.h>
int foo(int x);
int bar(int y);
int main ()      {
        int x = 1, y = 2, count;
        for (count = 1; count <=2; ++count){
                y+=foo(x) + bar(x);
        printf("\n %d",y);
        }
}
```

```c
int foo (int x) {
        int y;
        y=bar(x);
        return(y);
}
int bar (int x) {
        static int y = 10;
        y+=1;
        return (y+x);
}
```

x 1

y 16 11 12

int y; 1

y=bar(x);

return(y);

1

return (y+x);

12+1

Count = 1

y = y + foo(1) + bar(1)

2 + 12 + 13

27

HW

(A) 27 56 ✓ (B) 42 74 (C) 33 37 (D) 32 32

Count = 2 27 + foo(1) + bar(1)

Slide



# Recursion

* Recursion is problem Solving Technique in which Solution of a problem expressed in terms of itself

* In C. Language it a function which Call itself

function calling itself

Slide



# Recursion

In Recursive function the most important thing is base condition.

```
int main() {
    printf("I am a good student");
    main();
}
return 0;
```

main(); ← Recursive call

Slide



# Recursion




## Stack Runtime

Memory

* More memory Required

Stack overflow

abnormal Termination

Stack's allocated memory exausted (finish)

Slide



# Recursion

Base condition of Recursion, when reached Recursion Terminates



# Recursion

## Factorial

$$n! = n * (n-1)!$$

$3! = 3 * 2!$

$* 2 * 1!$

$1! = 1 \quad 0! = 1$ ← base condition

Slide



# Recursion

```
int fact(int n) {
    if(n==0||n==1)
        return 1;
    else
        return n* fact(n-1);
}
```

Base condition

Recursion

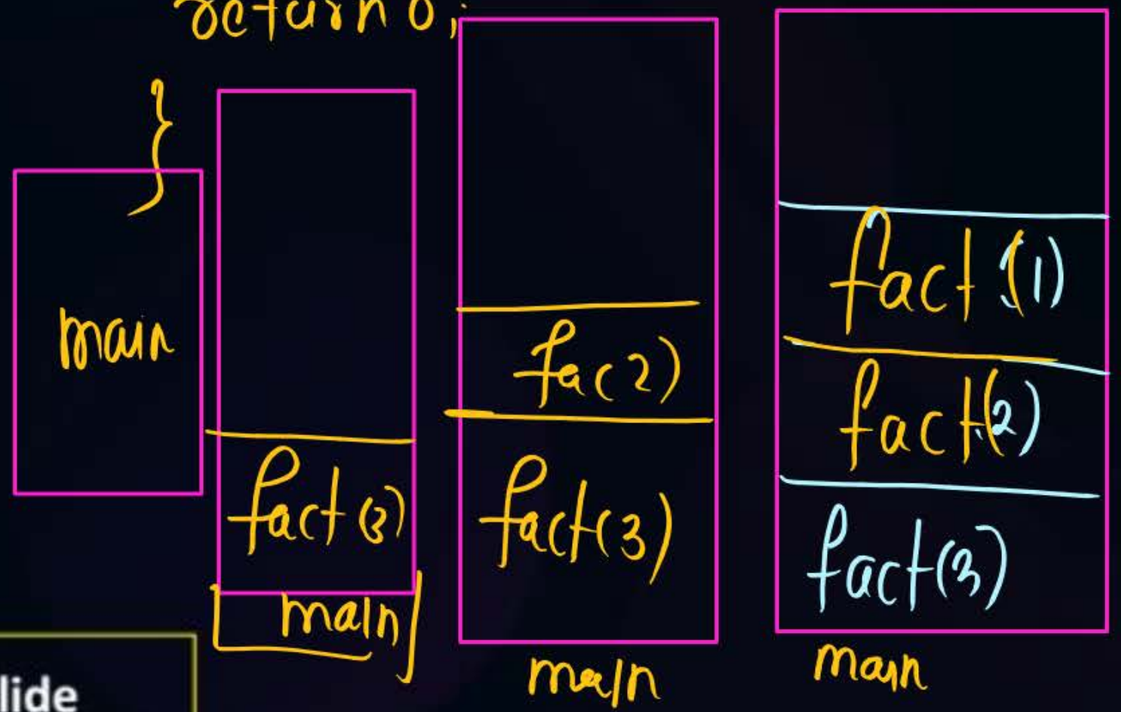
1. First check is for base condition

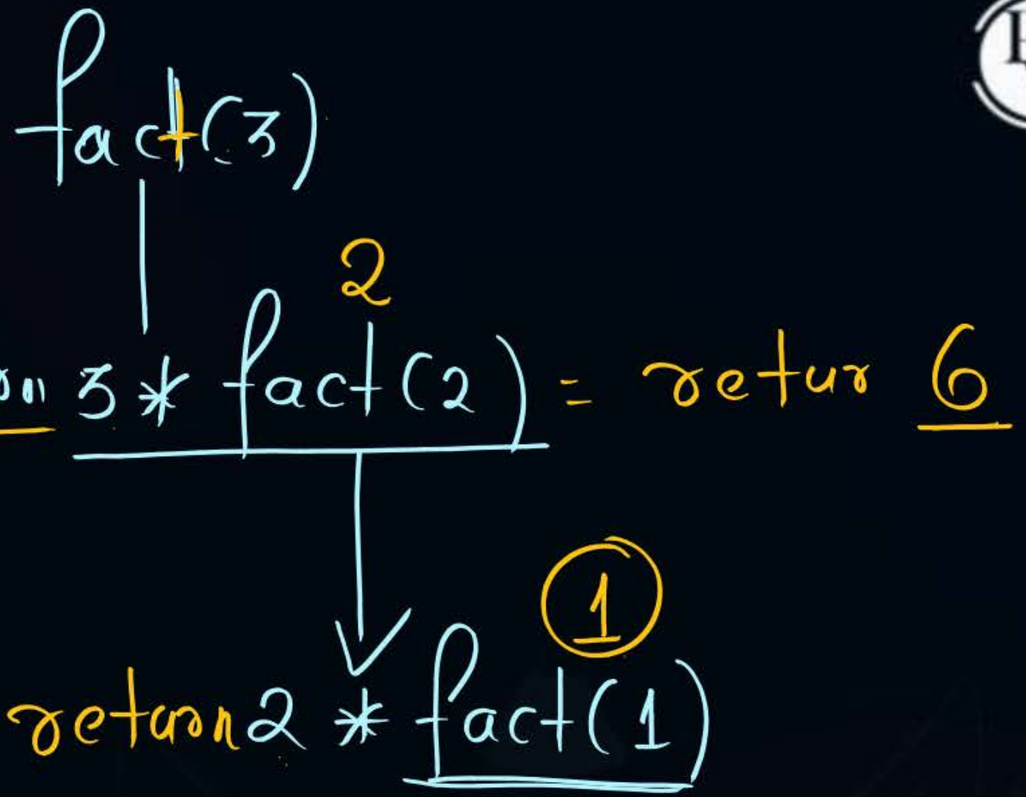
Slide



# Recursion

```
int main(){                    6
    printf("%d", fact(3));
    return 0;
}
```




Slide



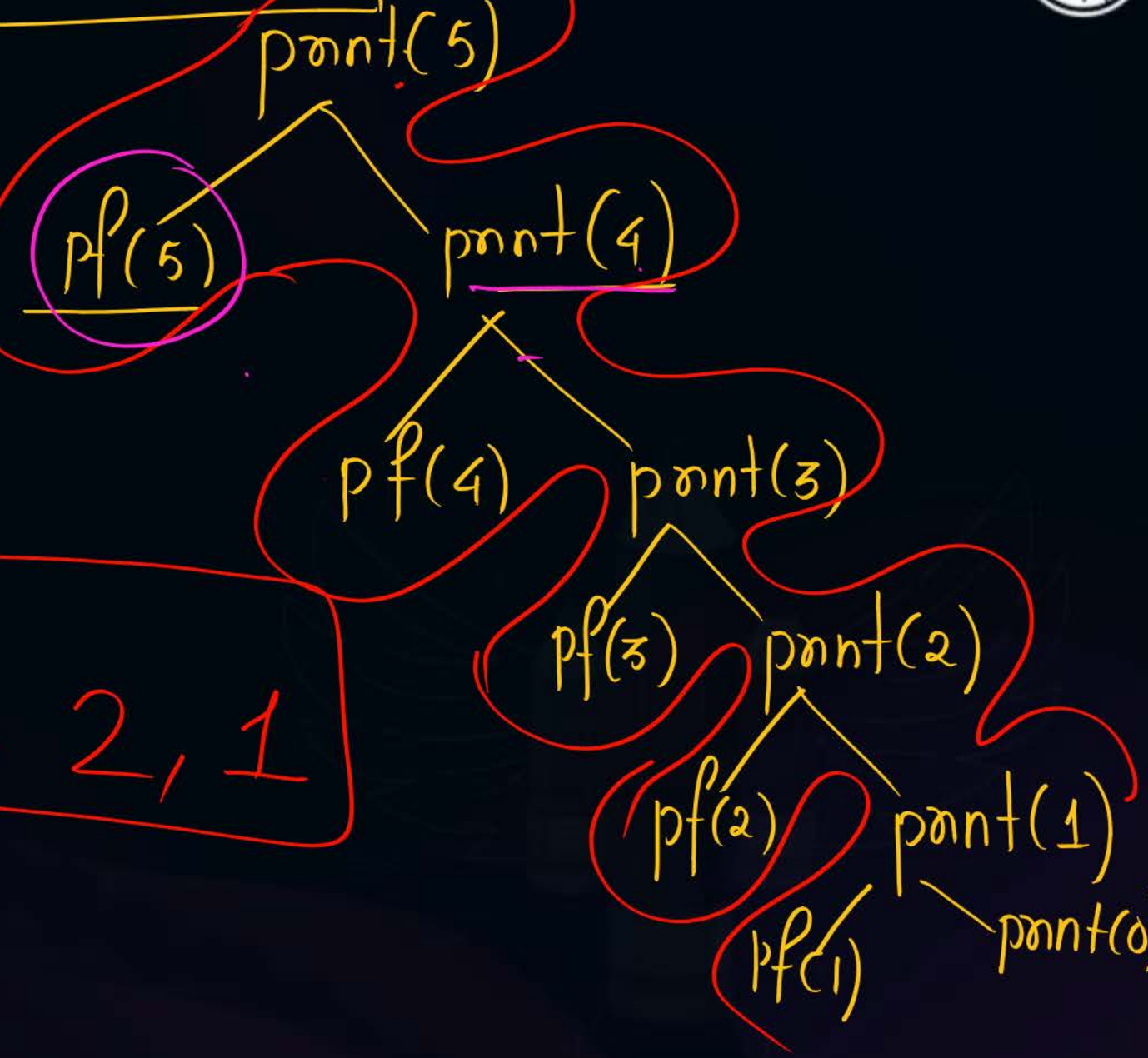
# Recursion

Recursion Tree

What is the output

```
#inalcude<stdio.h>
void print(int n){
      if (n <= 0) return;
      printf("%d", n);
      print(n-1);
}
int main(){
      print(5);
      return 0;
}
```

5 4 3 2 1



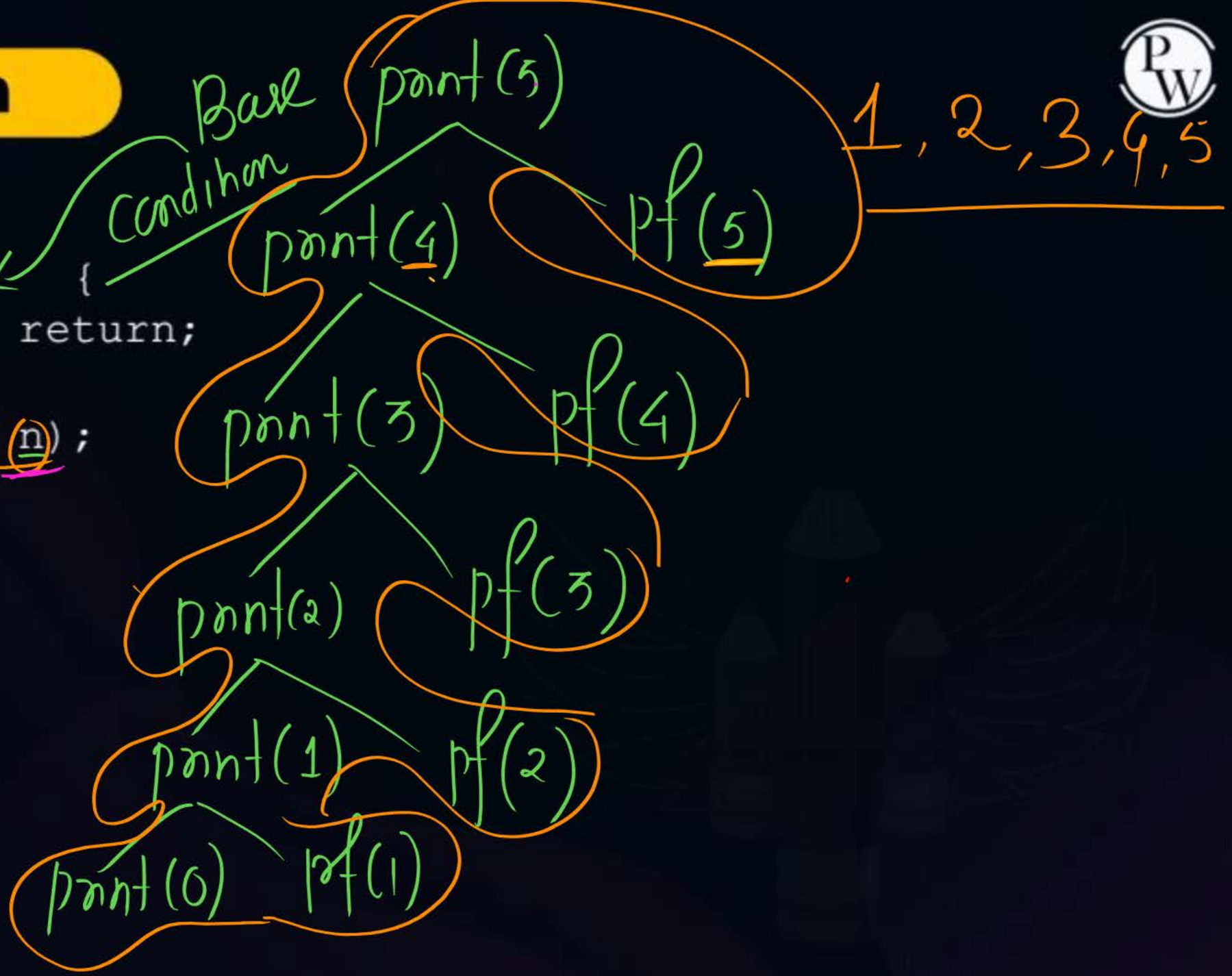

Slide



# Recursion

```
#include<stdio.h>
void print(int n)  {
      if (n <= 0)   return;
      print(n-1);
      printf("%d", n);
      }

int main(){
      print(5);
      return 0;
}
```

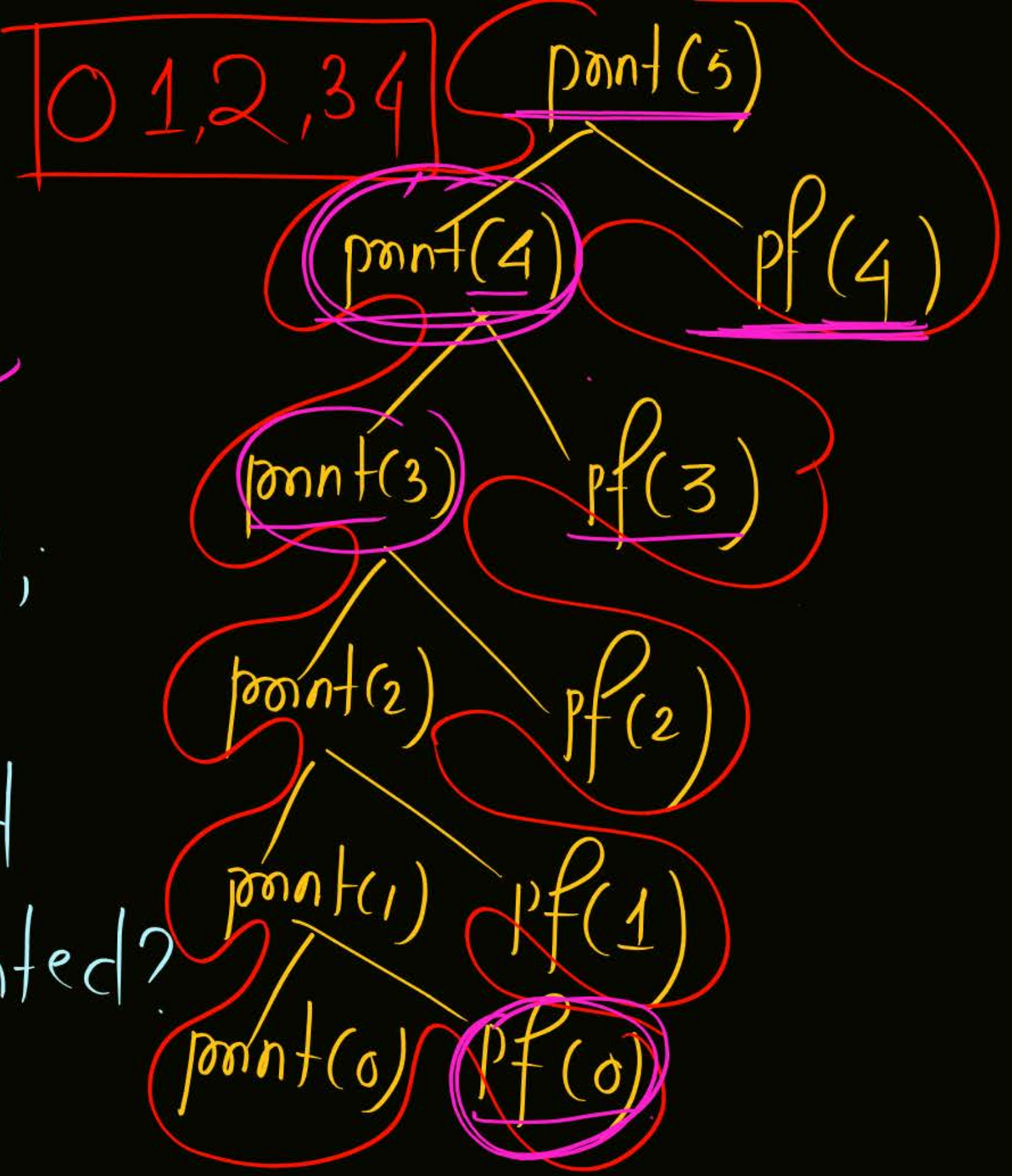



Slide

```
void print(int n){
    if(n<=0) return;
    print(--n);
    printf("%d", n);
}
```

n will updated predecrement update n.

0 1,2,3,4




if print(5) is called
what is the value printed?



# 2 mins Summary

Topic — Static variable problem

Topic — Recursion

Topic — Example factorial

Topic — 2 problem

Topic

Slide

# THANK - YOU

